Course: Cyber Security

Session: July-Dec 2026. Section-B

Assignment-2: Web Application Vulnerability Scenarios

Total no. of problems = 21

Total no. of groups = 21

Assignment Floated on: August 19, 2026

Seed-Date to be used: August 07, 2026

Instructions:-

Each group has been assigned one vulnerability scenario from the list below. You are expected to first study that vulnerability thoroughly — understand why it occurs and how it is typically exploited in real applications — and then design and build a small web application of your own, similar in spirit to the demonstrations of SQL Injection you just tried in the class, that deliberately contains this specific flaw. Your code must be uploaded to a GitHub repository and share the link. You will then record a screen-capture video with voiceover narration, no longer than 10 minutes in total, covering the full demonstration: first, run your application normally and briefly walk through its code and intended functionality, without revealing the vulnerability or hinting at how it will be exploited; next, demonstrate the actual attack live on screen against your own application, showing its effect clearly; then explain, in your own words, what the underlying vulnerability is, the specific flaw in your code responsible for it, and how it could be fixed. Upload the completed video to the shared Google Drive, and include the codes to your GitHub repository with a readme referring to the video.

If you have doubts please feel free to mail me mentioning your group-ID and assigned problem no.

Upload your codes and demonstration only on the authorized Github Classroom and Google Drive (TBA).

Set of Problems:-

1. **Stored Cross-Site Scripting (XSS)**
   Happens when an application saves user-submitted content — a comment, a bio, a review — and later displays it to other users without neutralizing it first. If that content contains actual HTML or JavaScript, the browser doesn't just show it as text, it runs it. Because the payload is permanently stored, no crafted link or trick is needed to reach victims; it fires automatically for everyone who later views that content, potentially including an administrator with far more access than the attacker.

2. **Cross-Site Request Forgery (CSRF)**
   Exploits the fact that a browser automatically attaches a logged-in user's cookies to every request sent to a site, even ones triggered from a completely different page. An attacker builds a hidden form on their own page that, when visited, silently submits a request to the target site — and because the victim's browser attaches their valid login cookie, the target site sees what looks like a legitimate request it never realized was triggered without consent.

3. **Business Logic Flaw**
   A different kind of flaw altogether: not a coding mistake in the usual sense, but a faulty assumption about how a legitimate process should behave. A classic example is a purchase or credit form that accepts a quantity field, where nobody considered that a user could submit a negative number, turning a purchase into a refund. Nothing is bypassed or tricked here; the server does exactly what it was told — the flaw lies entirely in what the developer assumed a reasonable input would look like.

4. **Open Redirect**
   Happens when an application takes a URL from user input — often a parameter used to send someone back to a page after logging in — and sends the browser there without checking that the destination actually belongs to the same trusted site. An attacker crafts a link that starts at the real, trusted site (so it looks legitimate) but redirects on to a look-alike phishing page, exploiting the victim's guard being down because of where the link began.

5. **Path Traversal**
   Occurs when an application accepts a filename from user input to locate a file on disk, without properly restricting it to the intended folder. Using repeated “go up one directory” sequences in the input, an attacker can walk the path outside the intended folder and reach files elsewhere on the server that were never meant to be reachable through that feature.

6. **Session Fixation**
   Exploits an application that fails to issue a fresh session identifier at the moment of login. An attacker visits the site first, obtains a valid-looking session ID, then tricks a victim into using that exact same session ID before the victim logs in. When the victim authenticates, the server upgrades that same, attacker-known session to a logged-in one, and the attacker, still holding the identical ID, becomes logged in as the victim too.

7. **Excessive Data Exposure via API**
   Happens when a backend endpoint returns more data than the page actually needs — a profile endpoint returning a full internal record, including fields never shown on screen. Since a raw API response can always be inspected directly, any extra fields quietly included in it are exposed to anyone who looks, whether or not the interface itself displays them.

8. **Clickjacking**

Exploits the fact that a site can be loaded inside an invisible frame embedded within a different, malicious page. The attacker overlays a convincing fake button exactly on top of the real, invisible site's actual button underneath. The victim believes they are clicking the harmless-looking fake element, but they are actually clicking something on the real, trusted site, potentially confirming an action they never intended.

9. **Insecure Direct Object Reference (IDOR)**
   Happens when an application lets a user access a specific record — an invoice, a file, a profile — by referencing it directly, often just a number in a URL, without checking whether the current user is actually allowed to see that particular record. Change the number, see someone else's data. It is common precisely because it is easy to forget: the code fetches “a record” but never separately asks whether it is this user's record.

10. **CORS Misconfiguration**
    Browsers normally prevent a script on one site from reading responses from another, unless that other site explicitly permits it through specific headers. The vulnerability appears when a server sets an overly permissive policy, effectively telling every website on the internet that it may read this site's data using a visiting user's own logged-in session. An attacker's unrelated page can then quietly fetch data from the vulnerable site and genuinely read the response back.

11. **Verbose Error Disclosure**
    Occurs when an application, upon hitting an unexpected error, shows the raw technical details straight to the user — a full stack trace, the failed database query, internal file paths, software versions — instead of a generic message. None of this helps a legitimate user, but it hands an attacker a detailed map of exactly how the system is built, often pointing directly at what to try next.

12. **Reflected Cross-Site Scripting (XSS)**
    Happens when a page takes something from the current request, usually a URL parameter such as a search term, and displays it back without neutralizing it first. If that input contains actual HTML or JavaScript, the browser runs it rather than displaying it as plain text. Since the payload lives in the URL itself, an attacker crafts a malicious link and only needs a victim to click it.

13. **Insecure File Upload**
    Occurs when an upload feature does not properly verify what kind of file is actually being submitted, or where it ends up stored. If the server trusts the filename rather than genuinely checking the file's real content and type, an attacker can disguise an executable script as an ordinary file, upload it, and trick the server into running it later.

14. **Client-Side-Only Validation**
    Happens when a page checks something — a price, a required field, a limit — using only JavaScript running in the browser, without the server ever re-checking the same rule when the data actually arrives. Anyone can view, alter, or bypass browser-side JavaScript entirely, so any rule enforced only on the client can simply be ignored by sending the underlying request directly.

15. **Username Enumeration**
    A login, registration, or password-reset form leaks whether a specific username or email address exists in the system, often through a subtle difference in the response message or even response timing. Individually this feels harmless, but it lets an attacker silently build a confirmed list of real accounts to target with a later, more targeted attack.

16. **Mass Assignment**
    Occurs when a form accepts a whole object's worth of fields and saves all of them to the database without restricting which ones the client should actually be allowed to set. If a form visibly offers only one field, but the underlying update logic blindly accepts whatever is sent, an attacker can include extra fields the form never displayed and have them silently accepted.

17. **Sensitive Data Exposure**
    An unnecessary file — a database backup, a configuration file, an old data dump — is left reachable inside a web server's folder, even though nothing on the site links to it. Anyone who guesses or systematically searches for the path gains full access to whatever is inside, often including credentials that should never have been stored in a plainly readable form to begin with.

18. **Weak Rate Limiting / Brute Force**
    Happens when a login form allows unlimited attempts in a short period, with no lockout or delay after repeated failures. An attacker simply tries one password after another automatically and quickly, until one succeeds, turning what should be a difficult guessing problem into a fast, mechanical one.

19. **Cleartext Transmission of Sensitive Data**
    Occurs when a site sends credentials, session identifiers, or other sensitive data over an unencrypted connection. Without encryption in transit, anyone sharing the same network can capture the traffic and read the sensitive data as it travels, in plainly readable form, with no need to crack or guess anything.

20. **Missing Function-Level Access Control**
    Similar to referencing a specific record without permission, but applied to an entire feature rather than a single item. A sensitive action may be correctly hidden from a normal user's visible menu, but if the underlying code handling it does not separately verify the requester's permission level, anyone who discovers or guesses the relevant address can trigger it directly.

21. **Security Through Obscurity**

    Relying on secrecy — a hidden URL, an unlisted admin page, an unpublicized endpoint — as the only protection for something sensitive, instead of real authentication. The resource is not linked anywhere visible, so it feels hidden, but hidden is not the same as protected: anyone who discovers the URL, by guessing, a leaked link, or an automated scanning tool, gets straight in, since there is no actual gate checking who they are.

END